



PERTEMUAN 9

Object-Oriented Programming: **Inheritance**

Memahami konsep pewarisan dalam Java — bagaimana kelas baru dapat dibangun di atas fondasi kelas yang sudah ada untuk menciptakan hierarki yang efisien dan terstruktur.



Pengenalan Inheritance

Inheritance (pewarisan) adalah salah satu pilar utama dalam pemrograman berorientasi objek. Konsep ini memungkinkan seorang programmer untuk membuat kelas baru berdasarkan kelas yang sudah ada, sehingga kode tidak perlu ditulis ulang dari awal.

Pewarisan Anggota

Kelas baru (subclass) secara otomatis mewarisi atribut dan metode dari kelas induk (superclass), sehingga dapat langsung digunakan.

Percepat Pengembangan

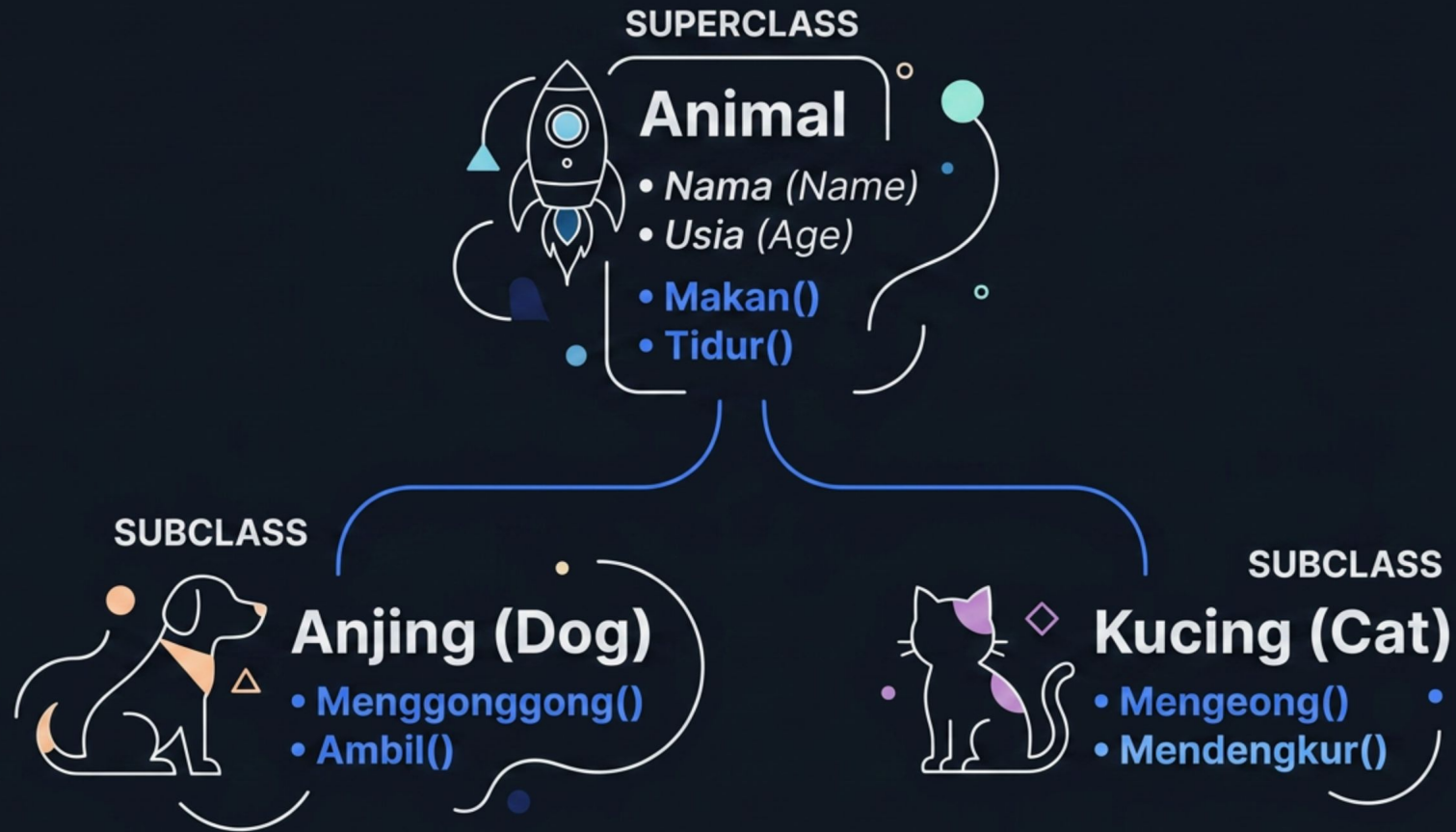
Dengan memanfaatkan kode yang telah terbukti benar, developer dapat fokus pada fungsionalitas baru tanpa mengulang logika yang sudah ada.

Mudah Dipelihara

Perubahan pada superclass secara otomatis berlaku pada seluruh subclass, meningkatkan efektivitas pemeliharaan sistem secara keseluruhan.

Superclasses dan Subclasses

Dalam Java, hubungan antara superclass dan subclass membentuk hierarki kelas. Superclass adalah kelas induk yang menyediakan atribut dan perilaku umum, sedangkan subclass adalah kelas turunan yang mewarisi sekaligus dapat memperluas atau mengubah perilaku tersebut.



Hubungan ini sering disebut sebagai relasi **is-a** — misalnya, seekor anjing *adalah* hewan. Konsep ini menjadi landasan dalam membangun sistem yang modular dan mudah dikembangkan.



Protected, Hubungan Superclasses dan Subclasses

Modifier protected

Menyediakan akses tingkat menengah — dapat diakses oleh superclass, subclass, dan kelas lain dalam paket yang sama. Modifier ini menggabungkan **inheritance access** dan **package-level access** secara bersamaan.

Studi Kasus: Hierarki Karyawan

- **Superclass — KomisiKaryawan:** Dibayar berdasarkan persentase dari total penjualan yang berhasil dilakukan.
- **Subclass — KomisiBergajiKaryawan:** Mewarisi seluruh perilaku superclass, namun juga menerima gaji pokok tambahan di atas persentase penjualan.
- Contoh nyata pemanfaatan pewarisan untuk memperluas fungsionalitas tanpa menulis ulang kode.

Membuat Kelas `CommissionEmployee`

Kelas `CommissionEmployee` merepresentasikan karyawan yang mendapat upah berdasarkan persentase komisi dari nilai penjualan. Berikut adalah struktur dan implementasinya.

1

Deklarasi Atribut

Atribut seperti `firstName`, `lastName`, `socialSecurityNumber`, `grossSales`, dan `commissionRate` dideklarasikan sebagai `private` untuk menjaga enkapsulasi data.

2

Konstruktor

Konstruktor menerima semua parameter yang diperlukan dan melakukan validasi awal, misalnya memastikan nilai `commissionRate` berada dalam rentang yang valid (0.0 – 1.0).

3

Getter & Setter

Setiap atribut dilengkapi dengan metode getter dan setter yang memungkinkan akses terkontrol dari luar kelas, termasuk dari subclass yang mewarisi kelas ini.

4

Metode `earnings()`

Menghitung total pendapatan karyawan: `grossSales * commissionRate`. Metode ini dapat di-override oleh subclass untuk menyesuaikan logika perhitungan.

STUDY CASE

Menggunakan Kelas `CommissionEmployee`

Setelah kelas `CommissionEmployee` dibuat, kita dapat membuat objek dan memanggil metode-metodenya. Berikut adalah contoh penggunaan kelas tersebut dalam program utama.

```
CommissionEmployee emp = new CommissionEmployee(
    "Ana", "Putri", "123-45-6789", 10000.00, 0.06);

System.out.println("Nama      : " + emp.getFirstName() + " " + emp.getLastName());
System.out.println("SSN      : " + emp.getSocialSecurityNumber());
System.out.println("Penjualan : " + emp.getGrossSales());
System.out.println("Komisi   : " + emp.getCommissionRate());
System.out.println("Pendapatan : " + emp.earnings());
```

 Output: Program akan menampilkan informasi karyawan beserta total pendapatannya berdasarkan persentase komisi yang telah ditetapkan — dalam kasus ini sebesar 6% dari Rp10.000.

STUDY CASE

Menggunakan Inheritance Hierarchy

Kelas `BasePlusCommissionEmployee` memperluas `CommissionEmployee` dengan menambahkan gaji pokok (`baseSalary`) ke dalam struktur karyawan.

1 Subclass Baru

`BasePlusCommissionEmployee` memperluas `CommissionEmployee` dengan menambahkan atribut `baseSalary` dan menyesuaikan metode `earnings()`.

2 Relasi Is-A

`BasePlusCommissionEmployee` adalah `CommissionEmployee` — semua perilaku dan atribut superclass diwarisi secara penuh melalui mekanisme inheritance.

3 Kata Kunci `extends`

Kata kunci `extends` digunakan dalam deklarasi kelas untuk secara eksplisit menandakan bahwa pewarisan dilakukan dari superclass tertentu.

```
public class BasePlusCommissionEmployee extends CommissionEmployee {  
    private double baseSalary;  
    // konstruktor, getter, setter, dan override earnings()  
}
```

Akses Anggota **Private** di Superclass

Batasan Akses **private**

Anggota yang dideklarasikan sebagai **private** di superclass **tidak dapat diakses langsung** oleh subclass, meskipun subclass mewarisi kelas tersebut. Ini adalah mekanisme enkapsulasi yang dirancang untuk melindungi data internal kelas.

Solusi: Getter & Setter

Untuk mengakses atau memodifikasi atribut **private** dari superclass, subclass harus menggunakan metode **public** atau **protected** yang disediakan superclass.

- `getGrossSales()` — membaca nilai penjualan
- `setGrossSales(double s)` — mengubah nilai penjualan
- `getCommissionRate()` — membaca persentase komisi

❏ Pola getter/setter ini menjaga enkapsulasi sekaligus tetap memungkinkan subclass berinteraksi dengan data superclass secara aman.



STUDI KASUS

Hierarki Inheritance: `CommissionEmployee` → `BasePlusCommissionEmployee`



Struktur Pewarisan

`BasePlusCommissionEmployee` adalah subclass langsung dari `CommissionEmployee`. Seluruh atribut dan metode superclass diwarisi secara penuh.



Variabel `private`

Variabel instance di `CommissionEmployee` dideklarasikan sebagai `private` — tidak dapat diakses langsung oleh subclass mana pun.



Akses via Getter/Setter

Subclass menggunakan metode `public` atau `protected` untuk membaca dan memodifikasi variabel superclass secara tidak langsung.



Keuntungan Desain

Pendekatan ini menjaga prinsip enkapsulasi, memisahkan detail implementasi dari struktur pewarisan, dan meningkatkan keamanan data secara keseluruhan.

Contoh Kode: Implementasi Lengkap

Berikut adalah contoh implementasi lengkap dari hierarki inheritance antara `CommissionEmployee` dan `BasePlusCommissionEmployee`, termasuk override metode `earnings()`.

```
public class BasePlusCommissionEmployee extends CommissionEmployee {
    private double baseSalary;

    public BasePlusCommissionEmployee(String first, String last,
        String ssn, double sales, double rate, double salary) {
        super(first, last, ssn, sales, rate); // panggil konstruktor superclass
        setBaseSalary(salary);
    }

    public void setBaseSalary(double salary) {
        if (salary >= 0.0) baseSalary = salary;
        else throw new IllegalArgumentException("Gaji pokok harus >= 0");
    }

    public double getBaseSalary() { return baseSalary; }

    @Override
    public double earnings() {
        return getBaseSalary() + super.earnings();
    }
}
```

❗ Perhatikan penggunaan `super.earnings()` — subclass memanggil metode `earnings` dari superclass dan menambahkan gaji pokok di atasnya, bukan menulis ulang seluruh logika.

Output Program: Perbandingan Hasil

Dengan menjalankan kedua kelas dalam program utama, kita dapat melihat perbedaan perhitungan pendapatan antara `CommissionEmployee` dan `BasePlusCommissionEmployee`.

CommissionEmployee

```
Nama      : Ana Putri
SSN       : 123-45-6789
Penjualan : 10000.0
Komisi    : 0.06
Pendapatan : 600.0
```

Pendapatan dihitung murni dari persentase komisi penjualan tanpa gaji pokok.

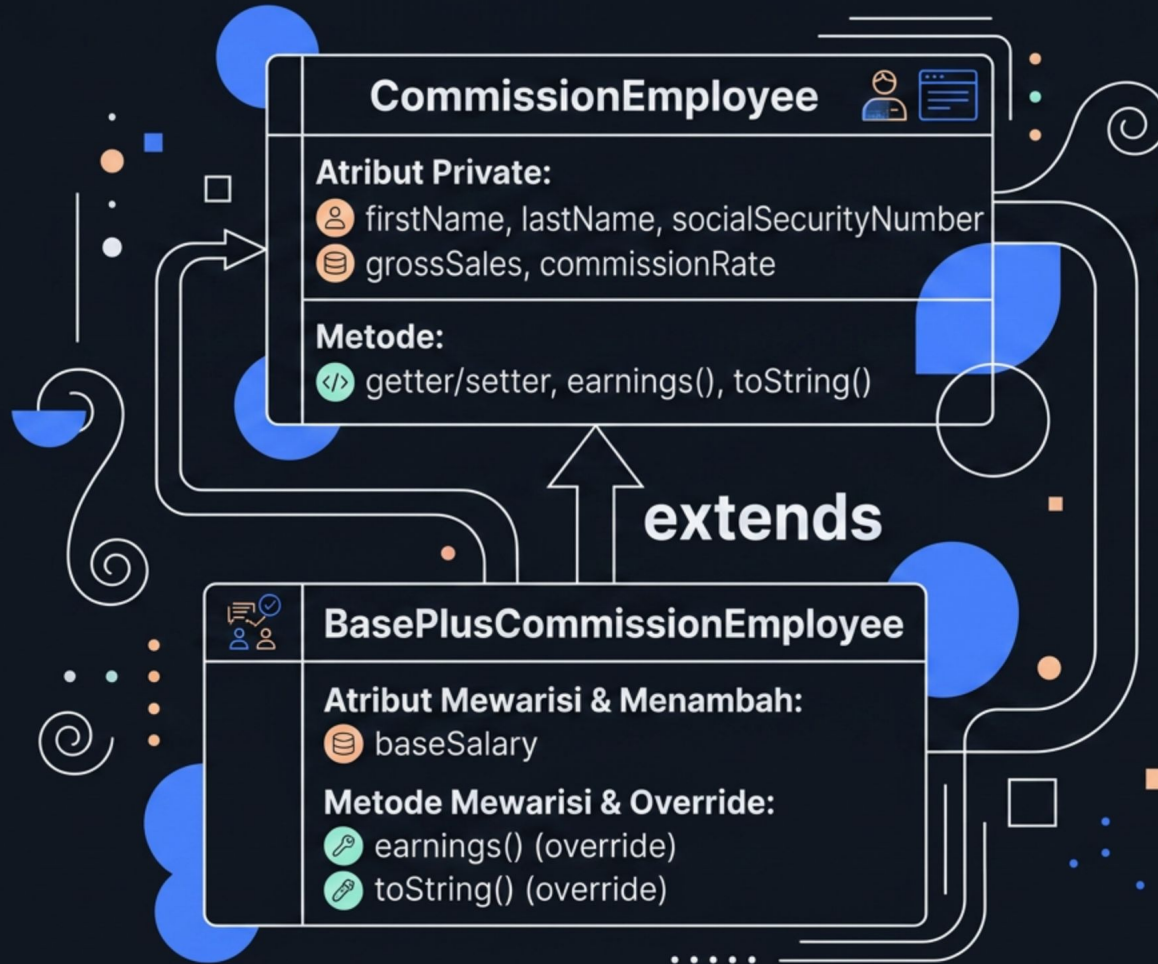
BasePlusCommissionEmployee

```
Nama      : Budi Santoso
SSN       : 987-65-4321
Penjualan : 10000.0
Komisi    : 0.06
Gaji Pokok : 3000.0
Pendapatan : 3600.0
```

Pendapatan total = gaji pokok + komisi penjualan, menghasilkan nilai yang lebih tinggi.

Visualisasi Hierarki Kelas

Diagram berikut menggambarkan struktur pewarisan lengkap antara `CommissionEmployee` sebagai superclass dan `BasePlusCommissionEmployee` sebagai subclass, beserta atribut dan metode masing-masing.



Relasi **is-a** antara subclass dan superclass memastikan bahwa objek `BasePlusCommissionEmployee` dapat diperlakukan sebagai objek `CommissionEmployee` di mana pun superclass diharapkan.

Constructors pada Subclasses



1

Instansiasi Subclass

Ketika objek subclass dibuat, proses instansiasi memicu **constructor chaining** — rantai pemanggilan konstruktor ke atas hierarki kelas.

2

Pemanggilan `super()`

Kata kunci `super()` digunakan secara eksplisit untuk memanggil konstruktor superclass dengan argumen yang sesuai. Jika tidak disebutkan, Java secara implisit memanggil konstruktor default (tanpa argumen) dari superclass.

3

Urutan Eksekusi

Konstruktor superclass **selalu dijalankan terlebih dahulu** sebelum konstruktor subclass melanjutkan inisialisasi atribut miliknya sendiri.



Jika superclass tidak memiliki konstruktor default (tanpa argumen), maka subclass **wajib** memanggil `super(...)` secara eksplisit dengan argumen yang tepat — jika tidak, program akan gagal dikompilasi.

Metode pada `Class Object`

Di Java, semua kelas secara implisit mewarisi dari kelas `Object` — kelas akar dari seluruh hierarki kelas Java. Kelas ini menyediakan sejumlah metode dasar yang dapat di-override oleh setiap kelas turunan.

`toString()`

Mengembalikan representasi String dari objek. Sering di-override untuk memberikan output yang lebih informatif dan mudah dibaca.

`equals()`

Membandingkan dua objek berdasarkan isi (bukan referensi memori). Harus di-override bersama `hashCode()` untuk konsistensi.

`hashCode()`

Mengembalikan nilai integer yang merepresentasikan objek, digunakan dalam struktur data berbasis hash seperti `HashMap` dan `HashSet`.

`getClass()`

Mengembalikan objek `Class` yang merepresentasikan tipe runtime dari objek tersebut — berguna untuk reflection dan debugging.

Override `toString()` pada Subclass

Salah satu praktik terbaik dalam Java adalah meng-override metode `toString()` dari `Object` agar output objek lebih bermakna. Berikut adalah contoh penerapannya pada hierarki karyawan.

```
// Dalam CommissionEmployee:
@Override
public String toString() {
    return String.format(
        "Karyawan Komisi: %s %s%nSSN: %s%nPenjualan: %.2f%nKomisi: %.2f",
        getFirstName(), getLastName(), getSocialSecurityNumber(),
        getGrossSales(), getCommissionRate());
}

// Dalam BasePlusCommissionEmployee:
@Override
public String toString() {
    return String.format(
        "%s%nGaji Pokok: %.2f", super.toString(), getBaseSalary());
}
```

- ✔ Dengan memanggil `super.toString()`, subclass dapat memanfaatkan output superclass dan hanya menambahkan informasi baru — menghindari duplikasi kode secara elegan.

Composition vs. Inheritance

Meskipun inheritance sangat powerful, penggunaannya yang berlebihan dapat menciptakan hierarki kelas yang kaku dan sulit diubah. Composition menawarkan alternatif yang lebih fleksibel.

⚠ Overuse of Inheritance

Pewarisan yang berlebihan menciptakan ketergantungan ketat antar kelas. Perubahan pada superclass dapat berdampak tak terduga pada seluruh subclass — masalah yang dikenal sebagai **fragile base class problem**.

- Hierarki dalam yang sulit dinavigasi
- Sulit mengubah superclass tanpa merusak subclass
- Relasi is-a tidak selalu tepat

☑ Keunggulan Composition

Composition (*has-a* relationship) lebih fleksibel — sebuah kelas menggunakan objek dari kelas lain sebagai atribut, bukan mewarisinya. Ini mengurangi keterikatan antar kelas.

- Lebih mudah diubah dan diuji
- Mendukung prinsip *favor composition over inheritance*
- Kombinasi keduanya sering menjadi solusi terbaik



Terima Kasih!

Semoga materi **Object-Oriented Programming: Inheritance** pada pertemuan 9 ini bermanfaat dan dapat diterapkan dalam proyek pemrograman Anda.



Poin Utama

Inheritance memungkinkan reuse kode melalui hubungan superclass–subclass yang terstruktur dengan baik.

Akses & Enkapsulasi

Gunakan modifier `protected` dan getter/setter untuk mengakses data superclass dengan aman dari subclass.

Constructor Chaining

Selalu panggil `super()` secara eksplisit untuk memastikan inisialisasi superclass berjalan dengan benar.

Desain Cerdas

Gunakan inheritance secara bijak — kombinasikan dengan composition untuk desain sistem yang fleksibel dan maintainable.